OrchLang: Auditing Resource Contracts for Bounded LLM Workflows*

Nambi Rajan M[a], Ranjithkumar S[b,*]

[a]*Department of Computational Intelligence, SCOPE, Vellore Institute of Technology, India*
[b]*Department of Analytics, SCOPE, Vellore Institute of Technology, India*

Abstract

OrchLang is an experimental compiler and audit artifact for bounded workflows using language models. It separates synthetic resource bounds, request equality and execution assumptions. An audit found an accepted workflow whose secret branches consumed 2 and 987 mock units. The repaired implementation distinguishes bindings from semantic contracts and retains request content. Separate readers reconstruct closed and response-dependent fragments for Lean checking. A text-valued core covers downstream requests under shared public inputs, secret Boolean control and stateless kernels. Reproduction includes 576 closed-request programs, 256 pipelines with 3,072 execution contexts, tokenizer probes and 11 documented external cost projections. These support scoped engineering claims, not provider privacy or a new general cost logic.

Keywords: resource analysis, information flow, workflow languages, certificate checking, reproducibility

*Author-review draft, 24 October 2026. Public archival release and final author declarations are pending. Not submitted.

*Corresponding author.

Email addresses: nambirajan.m2024@vitstudent.ac.in (Nambi Rajan M), ranjithkumar.s@vit.ac.in (Ranjithkumar S)

Metadata

Nr.	Code metadata description	Please fill in this column
C1	Current code version	Publication audit candidate; current core `00420ef`; earlier results `6f325be` and `7a0095d`. Full hashes in manifests.
C2	Permanent link to code/repository used for this code version	Project repository. Tested revision supplied locally; archival release pending.
C3	Permanent link to Reproducible Capsule	Not yet deposited. Local source bundles and raw records supplied with draft.
C4	Legal Code License	MIT (software); LPPL for Elsevier template derivative. Third-party notices included.
C5	Code versioning system used	Git
C6	Software code languages, tools, and services used	C++17, Python, Lean 4.19.0; optional Tokenizers 0.21.4. No inference service.
C7	Compilation requirements, operating environments and dependencies	CPU, C++17 compiler, Python 3.10+. Tested: GCC 16.1, Python 3.14.3, Windows 11. Tokenizer probe: Python 3.12.14.
C8	If available, link to developer documentation/manual	Repository README and included `docs/REPRODUCIBILITY.md`, `SEMANTICS.md`, `CERTIFICATES.md`.
C9	Support email for questions	`ranjithkumar.s@vit.ac.in`

Table 1: Code metadata. Public deposition remains a release requirement.

1. Motivation and significance

A resource bound limits consumption; it need not make consumption independent of a secret. A proof about a request kernel also requires an adapter that serializes that request and accounts for billed failures. OrchLang provides a small executable setting for inspecting and falsifying such assumptions.

The original checker at commit `a49fcdb` accepted a secret conditional whose branches declared models named `m` with output caps 1 and 1000. It equated their signatures by name. With seed 1 and identical public pins, the branches consumed 2 and 987 synthetic units. All 95 original tests passed. This refutes the original acceptance guarantee, although the unary bound of 1001 covers both observations. The artifact retains the witness and raw logs.

Resource-aware noninterference already combines information flow and cost analysis [1]. RelCost reasons about relative costs [2], and probabilistic AARA derives expected-cost bounds [3]. Randomness does not preclude numeric specifications. eRHL supplies a richer relational probabilistic logic [4]. Hard-KLAIM proves typed trace safety under interface assumptions [5]; PC2 checks constraints before operations and identifies conditions for real-world correspondence [6]. Loom's author-hosted report addresses structural LLM workflow cost bounds [7]; its peer-review status and artifact results are not established here. OrchLang claims no expressiveness or priority separation. Its software contribution is an inspectable specialization with counterexamples, repaired identity tracking, a separate certificate path and reproducible validation.

2. Software description

2.1. Architecture and functionality

The C++ frontend supports bounded inputs, prompts, lexical models, conditionals, bounded retries, outputs and tool sinks. Analyses track confidentiality, integrity and synthetic resource bounds. Resolved binding identities prevent shadowing errors. Canonical abstract contracts admit equivalent aliases while distinguishing caps and model revisions. Relational signatures retain template text, ordered typed arguments, literal content and resolved public guards. A secret branch is conservatively rejected when its request relation cannot be certified.

Commands `check`, `ir-json`, `certify` and `run` produce diagnostics, structural IR, an analysis report and length-only mock execution. Despite its historical name, `certify` emits an explicitly labelled report. It is not independently checked proof evidence. Numeric parsing rejects malformed pins and overflow; runtime pins must respect declared bounds. Arithmetic uses safe division and saturation, with saturated static totals rejected.

Separate certificate paths independently reparse source, reconstruct request expressions and check source digest/core agreement before invoking Lean on an untrusted plan. The original path admits closed requests. The new dataflow path also admits shared public text/Boolean inputs and response bindings used by later requests. Source-local names are aligned to lexical slots. Secret text, source retries, tools and reclassification fail closed. Dataflow outputs must be root literal statements. The Python readers and slot allocation are trusted, not mechanically verified. Neither path checks the C++ mock budget as a tokenizer budget.

Source ⟶ C++ analysis/report and length-only mock.

Same source, restricted fragments ⟶ independent Python readers and digest binding ⟶ JSON cores and untrusted plans ⟶ Lean decoder/checker.

Hand-authored text core ⟶ finite reference interpreter ⟷ separate finite-table replay executor.

Trusted boundaries: source reader/serialization, JSON library, primitive contracts, tokenizer library and execution platform.

Figure 1: Separate evidence paths. C++ acceptance is not a premise of the Lean theorem; no verified translation covers all source features.

2.2. Formal guarantees and assumptions

The unary Lean core bounds scalar cost for calls, sequence, branch maxima and up-to-n repetition, assuming primitive call bounds. The closed relational core uses a shared stateless kernel indexed by contract and complete request. The dataflow core additionally evaluates literal, slot and append expressions in a text store. A call binds actual response text; subsequent requests use that store. Its kernel $K(c, r)$ returns text, status and weight. Events record contract, UTF-8 input/output byte counts and failure status. Sequence composes stores, concatenates traces and multiplies weights. Lexical bounded retry restarts from the outer store, retains failures and stops after an attempt with no failed call. Errors are status values; the body completes before retry decides. This is not an SDK exception semantics or the C++ mock's independent failure sample.

Normalization removes a secret branch only when its arms have exactly the same plan. Lean's `normalize_correct` proves

$$N(P) = \mathrm{some}(Q) \implies \mathcal{D}_{K,l,h}(P) = \mathcal{D}^{\mathrm{plan}}_{K,l}(Q) \quad \text{for every Boolean store } h.$$

Here l is a shared public Boolean store. In the dataflow core the equality is between complete text-state transformers, for every shared initial text store. Thus secret Boolean stores yield equal weighted observations even when later requests depend on earlier response contents. A deterministic observer preserves equality. The result is algebraic; probability interpretation additionally requires normalized nonnegative finite kernels. Runtime checks validate the supplied tables, not a provider's compliance.

The executable decoder/checker has the theorem `decoded_certificate_sound` for successfully decoded core values. Lean 4.19.0 checks all three files without admitted lemmas or custom axioms; reported axioms are `propext` and `Quot.sound`. Complete source correspondence, secret text inputs and

stateful providers remain outside the proof. Hash binding detects source changes; it authenticates neither authorship nor the reader's correctness.

3. Illustrative examples

Requests `yes` and `no!` have the same byte length. A valid finite kernel may return one byte for the former and 100 for the latter. Input-length equality cannot justify equal output distributions; a common random-length tape would alter a marginal. The repaired checker retains content and rejects this example. Equivalent model aliases with equal constructed requests are accepted in `examples/valid/closed_certificate.orch`. The certificate manual provides executable issuance/checking commands.

A downstream witness returns `aa` or `bb` with equal probability. Their lengths agree, but requests `Next:aa` and `Next:bb` produce one and three output bytes. Replacing the response-dependent request with the constant `Next:aa` changes its distribution. The dataflow checker rejects that substitution and admits two matched pipelines with renamed local bindings. Its source example is in `examples/certificates/dataflow_certificate.orch`.

Ordered observations contain billed events. Aggregate observations sum usage, calls and failures by contract and unit. Reversing different calls can preserve the aggregate but change the ordered trace. Timing, provider-visible contents and tool effects are excluded. Equal bills do not authorize secret disclosure. For a call succeeding with probability 1/2, two-attempt retry yields success with weight 1/2, failure then success with weight 1/4, and two failures with weight 1/4. Failed attempts remain billed. Lean guards and replay observations check this example.

4. Impact and evaluation

OrchLang supports reproducible investigation of contract omissions. Evidence is engineering validation on a developer machine; independent human reproduction, production use and external adoption are not established. Clean local snapshots, source hashes, commands, raw observations and derived tables accompany the draft. CPU checks use GCC 16.1, C++17, warnings enabled and optimization level 2. No GPU, model inference or paid API was used. All counts below are generated from retained records by `scripts/report_publication.py` and `scripts/report_dataflow.py`.

The current suite passes 106 C++ and 47 Python tests. Four compiled semantic mutants are detected: name conflation, request-content erasure, missing retry multiplication and unbilled failure. A structural baseline using

maximum, sum and retry multiplication agrees on 23 existing cost workflows and 115 mock runs. It shares frontend primitive facts, which it cannot independently validate. Historical baselines omitting retries are retained only as broken negative controls.

Exhaustive development enumeration combines 24 arm variants into 576 programs, with four call-count structures. The compiler accepts 24, with 0 counterexamples under the supplied finite kernels. Decisions match conventional trace identity exactly. Sixteen rejected programs are equivalent under those particular kernels; they are not universally safe. No precision advantage is established. The separate source reader and Lean path perform 624 checks, issue 24 certificates and reject 48 source changes, 24 altered plans and four malformed batches. Reference and replay observations agree on 12 cases comprising 48 weighted paths. Enumeration is not a universal soundness proof.

The new development grammar covers 256 source programs with one two-call-per-arm pipeline shape and a public optional tail. Three text inputs, two public flags and two secret values give 3072 contexts. The independently constructed expected cores match the source reader throughout. Python and Lean normalization agree; Lean execution matches reference event distributions and replay request/event distributions in every context. The normalizer admits 16 programs, with 0 supplied-kernel counterexamples. Eight additional core cases, comprising 23 paths, cover UTF-8, nested/zero retry and the downstream witness. Tests reject 32 changed sources, 16 wrong plans and 8 invalid evaluator inputs.

The C++ checker accepts 8 of those source cases and emits E230 in 192: its PC label marks branch-local responses secret. The dataflow resource judgment does not override that confidentiality policy; the counts are not a precision ranking. Initial campaign attempts failed on a transport-size guard and unused prompt parameters. Batching and zero-argument fixed prompts repaired them without changing expected requests/cores or kernels. Failed records and amendments are retained. This is development validation, not held-out evaluation or a new general logic.

Tokenizer development probes use pinned GPT-2 BPE and BERT WordPiece files with Tokenizers 0.21.4. Of 24 fixed encodings, 13 exceed the estimate $\lceil b/4 \rceil$ for b UTF-8 bytes; 6 of 10 concatenations differ from summed component counts. All token IDs and special-token options are retained. These cases refute a general tokenizer-bound interpretation, not estimate a failure rate. The C++ unit is explicitly `mock-length-v1`.

An external census considers 15 Prompt Flow examples and six LangGraph tutorial patterns at pinned revisions [8, 9]. Eleven authored cost projections are included; ten exclusions have recorded reasons. Three basic

configurations share a call structure. One helper named `llm_result` returns a fixed string upstream and contributes zero model calls. Projections introduce bounded inputs/caps, abstract guards, summarize prompts and may serialize concurrency or omit aggregation. They preserve only documented cost shape: zero original applications or faithful end-to-end translations were executed. The corpus has no secret branches and supplies no relational precision evidence.

Projection	Calls	Input	Output	Total	Runs
pf-basic	1	260	120	380	20
pf-basic-with-builtin-llm	1	260	120	380	20
pf-basic-with-connection	1	260	120	380	20
pf-customer-intent	1	392	256	648	20
pf-ner	1	533	64	597	20
pf-switch	1	262	128	390	20
pf-if-else-stubs	0	0	0	0	20
pf-web-classification	2	2203	256	2459	20
lg-prompt-chaining	3	402	192	594	20
lg-parallelization	3	786	192	978	20
lg-routing	2	522	128	650	20

Table 2: External cost projections in synthetic mock units. Compiler and structural bounds agree throughout. Changes/exclusions are in the manifest.

The protocol fixes seeds 1–20 and bound/call-count endpoints. Its initial run failed on a reserved identifier in one projection; this was corrected and an amendment frozen before the complete rerun. Partial-record persistence was also repaired. The failure record is retained; this is not untouched preregistration. The amended run completes 220 mock executions with 0 componentwise violations and 0 baseline disagreements (Table 2). Repeated seeds are not independent application samples. No significance, throughput or provider-conformance claim follows.

5. Conclusions

Confusing names or lengths with semantic requests can produce false resource certifications despite passing tests. OrchLang makes this failure reproducible and supplies repaired analysis, separate source-bound plans, scoped Lean results and replay checks. Evidence supports synthetic bounds and text-dataflow equality under explicit contracts. General source verification, stateful providers and inference adapters remain open. Measuring annotation effort and retained functional behavior in independently adapted workflows is a next evaluation; the current projections do not answer that question.

Data, software and research-assistance statements

Source, raw records, protocols, claim ledger, reproduction instructions and tested-source bundles accompany the draft. Private library PDFs, credentials and model weights are excluded. A public versioned archive remains required before submission. The author reports no funding. Competing-interest declarations and both authors' final approval await confirmation.

OpenAI Codex assisted code, proofs, literature retrieval, scripts and writing. Its research-method role included constructing and testing the artifact. Both authors must verify the code, citations, proofs, results and text before submission; completed human review is not asserted.

References

[1] V. C. Ngo, M. Dehesa-Azuara, M. Fredrikson, J. Hoffmann, Verifying and Synthesizing Constant-Resource Implementations with Types, IEEE Symposium on Security and Privacy (2017). https://www.ieee-security.org/TC/SP2017/papers/408.pdf.

[2] E. Çiçek, G. Barthe, M. Gaboardi, D. Garg, J. Hoffmann, Relational Cost Analysis, POPL (2017). https://doi.org/10.1145/3009837.3009858.

[3] D. Wang, D. M. Kahn, J. Hoffmann, Raising Expectations: Automating Expected Cost Analysis with Types, Proc. ACM Program. Lang. 4, ICFP, Article 110 (2020). https://doi.org/10.1145/3408992.

[4] M. Avanzini, G. Barthe, D. Davoli, B. Grégoire, A quantitative probabilistic relational Hoare logic, arXiv:2407.17127v2 (2025). https://arxiv.org/abs/2407.17127v2.

[5] M. Aldinucci, A. Bracciali, I. Colonnelli, D. Medić, A. Mulone, L. Roversi, M. E. Santimaria, Hard-KLAIM: Hardening MCP-Enabled Agentic Workflows with a Locality-Aware Type-and-Effect System for Trace Safety, Journal of Logical and Algebraic Methods in Programming, pre-proof (2026). https://doi.org/10.1016/j.jlamp.2026.101191.

[6] C. Ciliberto, A. Rudi, E. Tassi, Proof-Carrying Contracts: Commitments AI Agents Cannot Break, arXiv:2610.28484v1 (2026). https://arxiv.org/abs/2610.28484v1.

[7] K. Dickerson, Statically Bounding Resource Accumulation in LLM Agent Workflows: The Cost Case, author-hosted report, 18 May 2026. https://loom.technology/research/bounded-llm-cost/.

[8] Microsoft, Prompt Flow, standard examples, revision 33a68c9d3cf349d7fdfad1f41969af27543c0406, accessed 23 October 2026. https://github.com/microsoft/promptflow.

[9] LangChain, LangGraph workflows and agents documentation, revision ecda7f522a6ffd5faaabb843b581fe57cd5eb782, accessed 23 October 2026. https://github.com/langchain-ai/docs.